



Instituto Tecnológico
de Buenos Aires

TP Final Programación Orientada a Objetos

Docentes: Franco Roman Meola, Matias Eugenio Bagini, Christian German Baus,
Victor Dalmagro, Manuel Dizenhaus

Comisión S2

Informe - Grupo 09

Integrantes del grupo:

Mateo Andrioli: 64042

Santino Giambelluca: 64697

Franco Pedretti: 61085

Ignacio Andreotti: 64280

Fecha de entrega del informe: 08/07/25

Cambios en la Implementación

- **AppLauncher:** Las modificaciones hechas fueron para corregir un error de funcionamiento ya que necesitábamos una ventana de tamaño flexible con un tamaño mínimo definido.
- **AppMenuBar:** Las modificaciones hechas fueron para corregir el estilo ya que modularizamos las funciones para que se entienda mejor el funcionamiento.
- **MainFrame:** Las modificaciones hechas fueron para corregir un error de funcionalidad, ya que queríamos que se adapte para llenar todo el espacio de la ventana, dándole prioridad a nuestro cuerpo central.
- **PaintPane:** Las modificaciones hechas abarcan los tres aspectos: corregir errores de funcionamiento, mejora de eficiencia y estilo. Esto se debe a la incorporación de manejo de errores, la refactorización de métodos, la corrección de errores como el almacenamiento de propiedades (color/borde) por figura y la implementación de nuevas funcionalidades.
- **StatusPane:** No hubo ninguna modificación en esta clase.
- **CanvasState:** Las modificaciones hechas abarcan los tres aspectos: corrección de errores de funcionamiento, mejora de eficiencia y estilo. Esto se debe a que la clase ahora no almacena "Figure", sino "CanvasFigure" para separar responsabilidades, se modificó asimismo el Iterable para que en vez de crear una copia cada vez, devuelve un iterador directamente sobre la lista interna. Por otro lado, se agregaron nuevos métodos y funcionalidades, y se aplicó la lógica de Reutilización de Código.
- **Figure:** Las modificaciones hechas fueron para corregir un error de funcionamiento ya que en lugar de ser una interfaz, la clase pasó a ser una abstracta a la cual extienden todas las figuras.
- **Point:** Las modificaciones hechas fueron para corregir errores de funcionamiento y de estilo, ya que no se utilizaba la Encapsulación y Control de Acceso, aparte se agregaron implementaciones de interfaces para un mejor comportamiento.
- **Circle:** Las modificaciones hechas fueron para mejorar el estilo, al usar un getter en lugar de un campo directo y establecer jerarquía extendiendo a Ellipse, y corregir errores de funcionamiento, ya que ahora se adapta a la nueva jerarquía.
- **Ellipse:** Las modificaciones hechas fueron para mejorar el funcionamiento, añadiendo validación y corrección de datos de entrada, y se añadieron nuevos métodos que le dan comportamiento. Además hubo una mejora de estilo al usar métodos que encapsulan lógica repetida.
- **Rectangle:** Las modificaciones hechas fueron para mejorar el funcionamiento, añadiendo validación y corrección de datos de entrada, y se añadieron nuevos métodos que le dan comportamiento. Además hubo una mejora de estilo al usar métodos que encapsulan lógica repetida.
- **Square:** Las modificaciones hechas fueron para mejorar el estilo, al usar la jerarquía extendiendo a Rectangle, y corregir errores de funcionamiento, ya que ahora se adapta a la nueva jerarquía.

Funcionalidades agregadas

Para empezar en el frontend agregamos la clase JavaFXDrawingContext, la cual hace de adaptador conectando nuestra lógica de dibujo abstracto (clases de figuras como Rectangle, Circle, etc.) con la biblioteca de gráficos de JavaFX. Esto significa que las figuras no necesitan saber *cómo* dibujarse en pantalla; solo saben *qué* quieren dibujar.

Luego en el backend agregamos diversas clases a medida que fuimos necesitando para poder obtener un programa limpio y claro.

En primer lugar creamos la clase CanvasFigure, la cual hace de wrapper entre las figuras geométricas y la clase CanvasState, y se encarga de aplicar los efectos a dichas figuras.

En segundo lugar agregamos la clase Color, la cual utilizamos para crear una **abstracción de color propia** para el programa, para no depender de ninguna biblioteca gráfica específica.

En tercer lugar agregamos los enums EffectType, Border y LineCap, los cuales definen los tipos de efectos, los tipos de borde y los estilos posibles para los extremos de líneas dibujadas, respectivamente.

Por último agregamos las interfaces FigureFactory, Colorable, Drawable, Movable y DrawingContext, en las cuales definimos todos métodos que utilizamos en las clases.

Problemas durante el trabajo

Durante el trabajo nos encontramos con diversos problemas los cuales fuimos resolviendo a medida que íbamos construyendo nuestro código.

Para empezar el primer problema con el que nos topamos era la falta de conocimiento acerca del uso del javaFX, ya que durante la cursada no lo habíamos utilizado. Esto fue fácil de resolver ya que nos bastó con leer la bibliografía propuesta por la cátedra para poder ir aprendiendo dicho tema.

Luego nos fuimos topando con pequeños errores en el frontend como, por ejemplo, el hecho de que al achicar la pantalla de nuestro programa desaparecían elementos como columnas o barras enteras.

Asimismo nos topamos con pequeños errores en el backend como, por ejemplo, que al momento de hacer los efectos no sabíamos cómo guardarlos para que la figura sepa que los tiene activados. También al momento de crear una figura nos pasó que en lugar de dibujarse hacia el lado donde soltamos el mouse, se dibujaba en el sentido contrario, y esto pasaba porque no tomamos en cuenta el sentido de los ejes cartesianos.

Por último el mayor problema que tuvimos fue que, sin darnos cuenta, muchas de nuestras clases de backend implementaban código perteneciente al javaFX por lo que no era correcto. Esto se debe a que javaFX es un lenguaje puro y exclusivamente del frontend. Por lo tanto tuvimos que revisar todo nuestro código devuelta y corregirlo para poder separar correctamente el frontend del backend.



